# Improving Multi-Constraint Instruction Following in 1B-Parameter Language Models via RECAST Fine-Tuning

Shreyas Natarajan  Fnu Rinkle Rose Renny  Mark Aashish Lnu  Gayathri Ethiraj

*Masters in Data Science, University of Wisconsin–Madison*

{natarajan9, rinklerosere, mlnu5, gethiraj}@wisc.edu

STAT 453, Spring 2026 · Team 15

**Abstract**

Small language models (∼1B parameters) offer practical advantages in cost and inference speed yet consistently underperform on multi-constraint instruction-following tasks. We present an end-to-end pipeline—9-stage data preprocessing and augmentation (30K → ∼90K examples), supervised fine-tuning via full parameter updates and LoRA, hybrid rule-based and LLM-judge evaluation, and 5-fold cross-validation—to quantify and improve constraint-following in 1B-parameter models using the RECAST-30K dataset [3]. Evaluating Llama 3.2 1B Instruct, the zero-shot baseline achieves a Hard Constraint Satisfaction Rate (CSR) of 28.9%. LoRA fine-tuning (rank 8, targeting `q_proj` and `v_proj`) reaches a 5-fold mean CSR of **80.0%**, substantially outperforming full fine-tuning (33.2% ± 2.3%). Length adherence (39.2%) and keyword inclusion (72.7%) remain the hardest constraints; structural and qualitative categories (format, tone, style) reach near-perfect pass rates after LoRA training.

## 1 Introduction

Large language models are powerful but expensive to deploy at scale. Models in the 1–2B parameter range offer substantially lower compute cost and faster inference latency, yet they struggle with *constraint-following*: simultaneously satisfying explicit requirements on format, length, tone, style, keywords, and structure imposed by a prompt. This difficulty compounds as the number of simultaneous constraints grows, a setting that directly mirrors real-world scenarios such as drafting structured reports, composing constrained marketing copy, or generating formatted API responses.

Benchmarks such as IFEval [1] and FollowBench [2] have documented these failures comprehensively. As a reference point, Llama-3.2-3B achieves only ∼77.4% prompt-strict accuracy on IFEval; the 1B variant is weaker still. The RECAST framework [3] addresses the data side by synthesizing multi-constraint training sets with automated validators enabling fine-grained, constraint-level supervision. RECAST fine-tuning at 7–8B scale outperforms much larger instruct-tuned models, but behavior at 1B scale is unexplored. Separately, LoRA [4] and full fine-tuning have been studied for reasoning and code tasks, but not specifically for constraint-following [5].

**Research questions.** This project addresses three open questions:

1. Can RECAST-based fine-tuning meaningfully improve constraint-following in 1B-parameter models?
2. Does full fine-tuning outperform LoRA, or do low-rank updates suffice?
3. Which constraint categories benefit most, and which remain difficult?

## 2 Related Work

**Constraint-following benchmarks.** IFEval [1] introduced a structured evaluation suite covering 25 verifiable instruction categories. FollowBench [2] extended this with multi-level difficulty constraints, showing sharp degradation as constraint count increases. Both benchmarks diagnose the problem but do not provide training recipes.

**RECAST.** Guo et al. [3] propose a constraint back-translation framework that synthesizes multi-constraint training data by extracting constraints from existing high-quality responses and injecting them back into prompts, guaranteeing physical achievability. Every constraint has an automated validator, enabling fine-grained training signal. At 7B–8B scales, RECAST outperforms larger instruct-tuned models; behavior at 1B parameters is unexplored.

**LoRA vs. full fine-tuning.** Hu et al. [4] introduced Low-Rank Adaptation, which freezes the base model and injects trainable rank-$r$ matrices into attention projections, training only ∼0.5% of parameters. Shuttleworth et al. [5] show that full fine-tuning adjusts existing singular vectors more conservatively while LoRA introduces "intruder dimensions" that can accelerate catastrophic forgetting—but this analysis targets reasoning and code tasks. Whether these dynamics hold for constraint-following remains open. Table 1 summarizes

the practical tradeoffs.

**Table 1:** LoRA vs. full fine-tuning at 1B scale. Both methods are novel for the constraint-following setting.

| Dimension | LoRA | Full FT |
|---|---|---|
| Trainable params | ∼0.5% | 100% |
| VRAM peak | ∼7–9 GB | ∼16–20 GB |
| Storage/run | ∼30–60 MB | ∼2.4 GB |
| Forgetting risk | Lower | Higher |

**Instruction-tuning data.** The Tulu corpus [6] assembles diverse instruction-following examples from FLAN, ShareGPT, OpenAssistant, and other sources, providing broad coverage needed to prevent catastrophic forgetting when fine-tuning on the narrower RECAST constraint distribution.

# 3 Dataset and Preprocessing

## 3.1 RECAST-30K

The RECAST-30K dataset provides multi-constraint instruction–response pairs covering 15 constraint categories (Style, Topic, Length, Keyword, Language, Format, Tone, End_With, Start_With, Role-Playing, Numerical, etc.) with an average of ∼13.4 constraints per instruction and 90.1% human annotator agreement. Each record contains:

- `winner_prompt` , the multi-constraint instruction;
- `response_of_winner_prompt` , a constraint-satisfying gold response;
- `added_constraint` , dict mapping category to constraint descriptions;
- `rule_evaluate_dict` , structured validator parameters (word length bounds, required keywords, start/end strings).

## 3.2 Preprocessing Pipeline

The raw JSONL passes through a nine-stage pipeline:

1. **Language detection.** Non-English prompts filtered via `langdetect` (ISO-639-1 check on first 2000 chars).
2. **HTML/entity removal.** Tags and entities stripped by compiled regex.
3. **Emoji/control removal.** Emojis replaced with spaces; invisible Unicode and long symbol runs removed.
4. **Quality gates.** Prompts <15 words, <85% printable, or with empty responses dropped.
5. **Constraint integrity.** Validates `added_constraint` structure and count consistency.
6. **Stopword removal.** Applied to prompts only, after quality gates.
7. **Exact deduplication.** SHA-256 fingerprints on normalized prompts.
8. **Near-deduplication.** MinHash LSH (`datasketch`, Jaccard $\tau$=0.85, 128 permutations).
9. **Augmentation.** Two techniques expand cleaned ∼30K records to ∼90K: *lexical editing* via synonym replacement (+∼30K) and *back-translation* via English→German→English round-trip (+∼30K).

A post-cleaning distribution audit flags categories below 50% of the expected uniform share. The pipeline runs in parallel with 8 workers.

## 3.3 Data Splits

For cross-validation, the ∼90K augmented dataset is partitioned into 5 folds with a 72% / 18% / 10% train/-validation/test split per fold.

# 4 Methods

## 4.1 Full Fine-Tuning

The full fine-tuning pipeline makes all ∼1.24B model parameters trainable using HuggingFace `Trainer` with gradient checkpointing. Each record is formatted as a user/assistant chat turn via the Llama-3 chat template; prompt tokens are masked (`labels`= $-100$) so loss is computed only on the response.

**Loss:** Standard cross-entropy over response tokens. An earlier custom penalty ($1-s/\text{total}$) encountered backpropagation difficulties; CE loss provided stable gradient-based optimization and is a principled probabilistic training objective under teacher forcing.

**Hyperparameters:** AdamW optimizer; lr = $2\times10^{-5}$; cosine LR scheduler with 3% warmup; effective batch 32 (per-device batch 4, grad accum 8); 2 epochs; max 512 tokens; bfloat16 on A100 40 GB; gradient checkpointing enabled.

## 4.2 LoRA Fine-Tuning

The LoRA pipeline applies rank-$r$ adapter matrices to `q_proj` and `v_proj` in all attention layers:

$$W' = W + \frac{\alpha}{r} \cdot BA \tag{1}$$

where $W \in \mathbb{R}^{d\times d}$ is frozen, $A \in \mathbb{R}^{r\times d}$ and $B \in \mathbb{R}^{d\times r}$ are trainable, $r$=8, $\alpha$=16, dropout = 0.05. This trains 81,500 adapter parameters (≈0.007% of weights); the adapter (∼30–60 MB) is saved while the base model remains frozen.

The pipeline uses `trl.SFTTrainer` with the Llama-3 chat format:

```
<|begin_of_text|><|start_header_id|>user<|
    end_header_id|>
{instruction}<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
{response}<|eot_id|>
```

**Hyperparameters:** lr $= 10^{-4}$; cosine scheduler; 10 warmup steps; weight decay 0.01; effective batch 32 (batch 8, grad accum 4); 1 epoch; max 512 tokens; bfloat16.

### 4.3 Evaluation: Constraint Validation

**Rule-based mode.** Four constraints are evaluated deterministically from `rule_evaluate_dict`:

**Table 2:** Rule-based constraint checks. Each returns 0 (fail), 1 (pass), or `None` (not applicable).

| Constraint | Check |
|---|---|
| `length` | Word count in $[\min, \max]$ |
| `keyword` | Each keyword appears $\geq$ required count |
| `start_with` | Response begins with required string |
| `end_with` | Response ends with required string |

**LLM-judge mode.** For qualitative constraints (style, tone, role-playing, numerical), a judge model (default: Qwen2.5-7B-Instruct; also Gemma-2-9B-Instruct at 4-bit) receives the response plus plain-English constraint descriptions and outputs a JSON verdict.

**Scoring metrics:**

$$\text{CSR}^{(i)} = \frac{\text{constraints satisfied in example } i}{\text{constraints checked in example } i} \quad (2)$$

$$\text{Hard CSR} = \text{all checked constraints satisfied} \quad (3)$$

The standalone `ConstraintChecker` class dispatches across 18 constraint types in 8 categories: length, keywords, structural, capitalization, format, language, punctuation, and detectable.

### 4.4 5-Fold Cross-Validation

The ∼90K augmented dataset is partitioned into 5 folds (72% / 18% / 10% train/val/test). The model is trained on $k-1$ folds and evaluated on the held-out fold, providing variance estimates for both LoRA and full fine-tuning under the same framework.

## 5 Experimental Setup

**Hardware.** All training ran on Google Colab Pro+ with NVIDIA A100 40 GB GPUs. Models were loaded in bfloat16.

**Models.** Primary model: **Llama 3.2 1B Instruct** (1.24B parameters, publicly available instruction-tuned causal decoder-only Transformer). Secondary (exploratory runs): Qwen 2.5 1.5B Instruct.

**Table 3:** Hardware requirements per experimental configuration.

| Run type | VRAM | Time | Params |
|---|---|---|---|
| LoRA (r=8) | ∼7–9 GB | 2–4 hr | 81,500 |
| Full FT | ∼16–20 GB | 4.5–8 hr | 1.24B |
| LLM judge (7B) | ∼14 GB | 30 min/500 | 7B |

**Evaluation protocol:** (1) zero-shot baseline on RECAST test split; (2) LoRA: merge adapter with base, evaluate on held-out fold per CV round; (3) Full FT: load checkpoint, evaluate on same fold; (4) per-category analysis across all training regimes.

## 6 Results

### 6.1 Zero-Shot Baseline

The zero-shot Llama 3.2 1B Instruct baseline achieves an overall Hard CSR of **28.9%** on the RECAST test set. Per-constraint pass rates (Table 4, Figure 1) reveal a strongly bimodal pattern: structural constraints (sentence length, format, capitalization, punctuation) are already satisfied near-perfectly by the pretrained model, while content-dependent constraints fail frequently. Hard CSR is low because even a single failure among the ∼13.4 average constraints per example counts as complete failure.

**Table 4:** Zero-shot baseline per-constraint pass rates for Llama 3.2 1B Instruct.

| Constraint | Pass rate | Notes |
|---|---|---|
| `word_length` | 32.0% | Hardest rule-based |
| `keyword` | 40.0% | Requires explicit counting |
| `start_with` | 56.5% | Partially structural |
| `end_with` | 36.0% | Hard to control endpoint |
| `sentence_length` | 100.0% | Trivially satisfied |
| `format` | 100.0% | Follows conventions |
| `all_caps/lower` | 100.0% | Easy to learn |
| `no_commas` | 100.0% | Rare; default satisfied |
| **Hard CSR** | **28.9%** | All must pass |

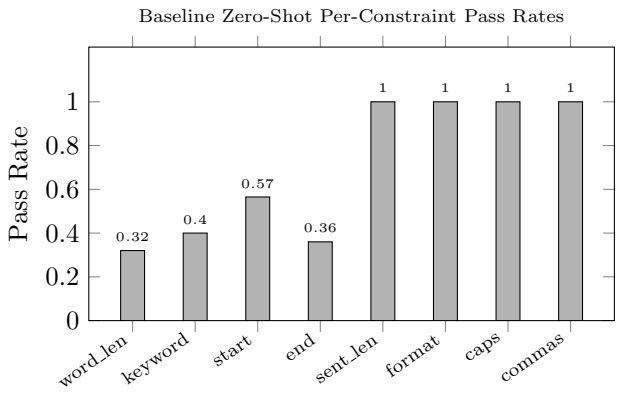


**Figure 1:** Zero-shot baseline pass rates. Content-dependent constraints (word count, keyword, start/end) score 32–57%; structural constraints are trivially satisfied at baseline.

## 6.2 LoRA Fine-Tuning Results (5-Fold CV)

LoRA fine-tuning achieves a 5-fold mean CSR of **80.0%** with very low fold-to-fold variance (Table 5), representing a +51.1 pp improvement over the zero-shot baseline.

**Table 5:** LoRA 5-fold CSR results (Llama 3.2 1B, $r$=8).

| Fold | CSR | N |
|---|---|---|
| 1 | 0.7972 | 200 |
| 2 | 0.7972 | 200 |
| 3 | 0.7967 | 200 |
| 4 | 0.8039 | 200 |
| 5 | 0.8050 | 200 |
| **Mean** | **0.8000** | 1000 |

Per-constraint pass rates (Table 6 and Figure 2) reveal a clear bifurcation: structural and qualitative constraints reach ceiling performance (≥99.7%), while length adherence (39.2%) and keyword inclusion (72.7%) remain challenging. Figure 3 confirms fold-to-fold variance is $< 1\%$, indicating stable generalization independent of the train/test split.

**Table 6:** LoRA per-constraint pass rates (mean ± std across 5 folds).

| Constraint | Mean | Std |
|---|---|---|
| `length` | 0.3920 | 0.0244 |
| `keyword` | 0.7270 | 0.0351 |
| `start_with` | 0.5830 | 0.0413 |
| `end_with` | 0.5020 | 0.0182 |
| `format` | 0.9970 | 0.0045 |
| `tone` | 0.9990 | 0.0022 |
| `style` | 1.0000 | 0.0000 |
| `role_playing` | 1.0000 | 0.0000 |
| `numerical` | 1.0000 | 0.0000 |

## 6.3 Full Fine-Tuning Results (5-Fold CV)

Full fine-tuning of all 1.24B parameters achieves a 5-fold mean CSR of **33.2%** (±2.3%), substantially below LoRA (Table 7). The abnormally low train loss in Fold 1 (= 0) likely reflects a logging artifact. Eval loss is stable across folds (±0.0024); the CSR range of 30.4–36.6% indicates greater sensitivity to data partitioning than LoRA.

**Table 7:** Full fine-tuning 5-fold results on Llama 3.2 1B.

| Fold | Eval loss | Train loss | CSR |
|---|---|---|---|
| 1 | 1.4217 | 0.0000 | 36.6% |
| 2 | 1.4215 | 10.9864 | 30.4% |
| 3 | 1.4177 | 10.9924 | 33.6% |
| 4 | 1.4251 | 10.9826 | 31.0% |
| 5 | 1.4229 | 10.9752 | 34.6% |
| **Mean** | 1.4218 | 8.7873 | **33.2%** |
| **Std** | 0.0024 | 4.3937 | 2.3% |

## 6.4 Head-to-Head Comparison

Table 8 and Figure 5 summarize results across all three training regimes. LoRA delivers a +51.1 pp gain over the zero-shot baseline and a +46.8 pp gain over full fine-tuning.

**Table 8:** Three-way model comparison. PR = per-constraint pass rate. Full FT per-constraint breakdown was not captured at evaluation time.

| Config | CSR | Len PR | Kw PR | Start PR |
|---|---|---|---|---|
| Baseline | 28.9% | 32.0% | 40.0% | 56.5% |
| Full FT | 33.2% | - | - | - |
| LoRA ($r$=8) | **80.0%** | 39.2% | 72.7% | 58.3% |

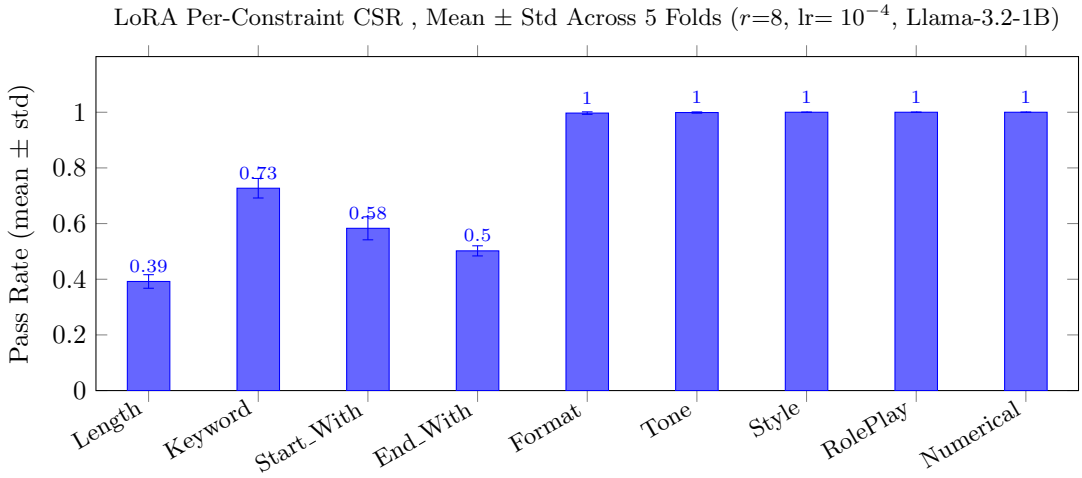


**Figure 2:** LoRA per-constraint pass rates with ±std error bars across 5 folds. Structural and qualitative constraints (format, tone, style, role-playing, numerical) reach near-perfect performance; length adherence (39.2%) remains the largest challenge.

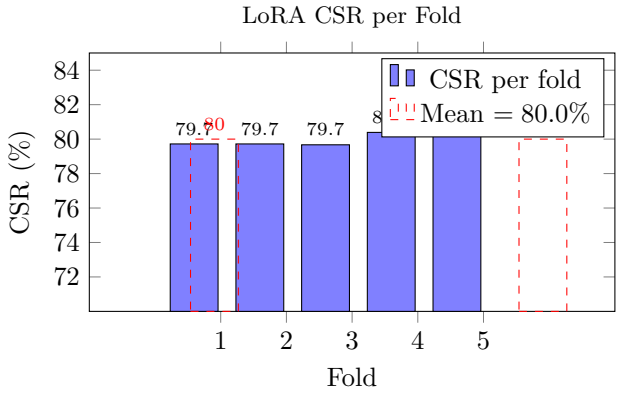


**Figure 3:** LoRA CSR per fold. Dashed red line marks the 5-fold mean (80.0%). Variance is $< 1\%$, confirming stable generalization.

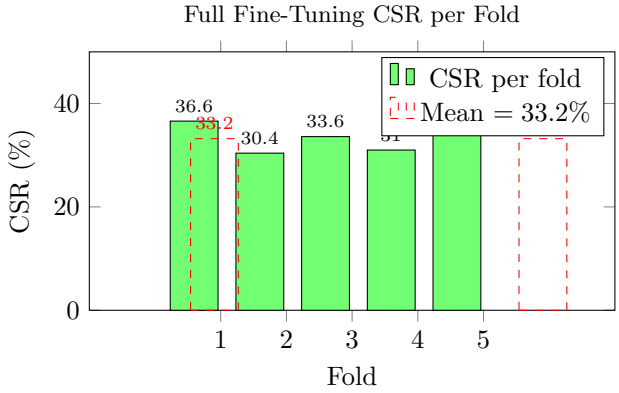


**Figure 4:** Full fine-tuning CSR per fold with mean line (dashed). Fold 1 shows the best performance (36.6%); variance is higher than LoRA.

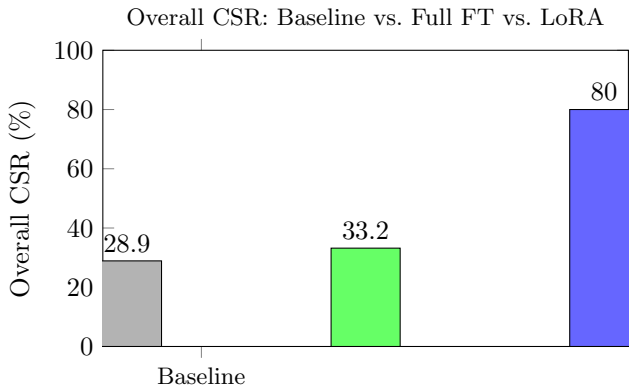


**Figure 5:** Overall CSR comparison. LoRA substantially outperforms both the zero-shot baseline (+51.1 pp) and full fine-tuning (+46.8 pp).

# 7 Discussion

## 7.1 Key Findings

**LoRA outperforms full fine-tuning by a large margin.** The 80.0% vs. 33.2% gap (+46.8 pp) is the most striking result and contradicts the hypothesis that full parameter updates should give the optimizer more expressive power. We attribute the reversal to *catastrophic forgetting*: updating all 1.24B parameters on the narrow RECAST constraint distribution tends to overwrite the broader instruction-following competence built into the pretrained checkpoint. LoRA's frozen base model preserves this competence while adapting attention projections to constraint-following patterns.

**Length adherence is the hardest constraint.** Despite a +7.2 pp gain (baseline 32.0% → LoRA 39.2%), word-count adherence remains below 40%. The model

must maintain an accurate running count of generated tokens while simultaneously satisfying other constraints and producing coherent text, a form of explicit bookkeeping that is difficult to learn from sequence-level supervision alone.

**Structural and qualitative constraints saturate.** Format, tone, style, role-playing, and numerical constraints all reach $\geq$99.7% pass rates under LoRA. These require a stylistic shift in the response distribution rather than explicit counting, and tend to be well-represented in the training data.

**Loss function design.** The custom penalty $(1 - s/\text{total})$, while conceptually intuitive, encountered gradient computation issues during backpropagation. Standard cross-entropy over constraint-satisfying gold responses proved more effective, as CE is a differentiable function.

### 7.2 Design Decisions

**Augmentation to ∼90K.** Tripling the training set via lexical editing and back-translation helps LoRA generalize beyond exact phrasings while preserving constraint semantics.

**5-fold CV.** The low fold-to-fold LoRA variance (CSR range: 79.67–80.50%) demonstrates robustness. The higher variance for full FT (30.4–36.6%) suggests full FT is more sensitive to training partition.

**Prompt masking.** Masking prompt tokens ensures CE loss is computed only over the response, preventing gradient dilution from the long constraint specifications embedded in RECAST prompts.

### 7.3 Limitations

1. **Model scale.** Resource constraints limited primary experiments to Llama 3.2 1B; the proposal targeted four 1–2B models. Conclusions about scale sensitivity are preliminary.
2. **Compute cost.** Given the student Google Colab tier of 200 compute units per month, we could not use the entire 90K augmented data points but rather only the 30K. Full FT required ∼4.5 hr/fold on A100, limiting hyperparameter exploration, and LoRA required ∼0.5 hr/fold. A100 chips use ∼7.25 CU/hr.
3. **Evaluation coverage.** The deterministic evaluator covers only 4 of 15+ constraint categories; the LLM judge adds inference cost and may introduce systematic bias (7B judge evaluating 1B outputs may differ from human annotators).
4. **Catastrophic forgetting.** We did not evaluate on IFEval after full fine-tuning; direct measurement of general instruction-following degradation remains future work.
5. **Full FT per-constraint breakdown.** Per-constraint pass rates were not captured for full FT, preventing direct per-category comparison between methods.

## 8 Conclusion

We built and evaluated a complete end-to-end pipeline for multi-constraint instruction-following in 1B-parameter language models, covering 9-stage data preprocessing with augmentation (30K → 90K), two fine-tuning regimes (full FT and LoRA $r$=8), hybrid evaluation (rule-based + LLM judge over 9 constraint types), and 5-fold cross-validation.

The zero-shot baseline achieves Hard CSR 28.9%, with length (32.0%) and keyword (40.0%) as the weakest categories. LoRA raises overall CSR to 80.0% (5-fold mean), with structural and qualitative constraints near ceiling. Full fine-tuning achieves only 33.2%, likely due to catastrophic forgetting of the pretrained instruction-following competence.

The central finding: **LoRA substantially outperforms full fine-tuning** for constraint-following at 1B scale. Training 0.007% of parameters outperforms training 100%, using half the VRAM, a fraction of the training time, and producing a 40× smaller checkpoint. This validates LoRA as the preferred adaptation strategy and motivates future investigation into higher ranks ($r$=16, $r$=32), targeted length-heavy augmentation, and multi-model robustness on Gemma 3 1B and SmolLM2 1.7B.

## Team Contributions

**Gayathri Ethiraj:** Designed and implemented the complete 8-stage dataset preprocessing pipeline, including constraint integrity validation and near-deduplication via MinHash LSH, and led overall system integration and the evaluation framework.

**Shreyas Natarajan:** Developed the back-translation and lexical editing augmentation strategies, contributed to overall system integration, and led training execution, result analysis, and visualization.

**Mark Aashish Lnu:** Conducted dataset clustering analysis, implemented the modular ConstraintChecker (18 constraint types), built the full fine-tuning pipeline with the constraint-aware loss, established the 5-fold cross-validation framework.

**Fnu Rinkle Rose Renny:** Built the LoRA fine-tuning pipeline, contributed to the 5-fold cross-validation framework, designed evaluation visualizations, and led per-constraint analysis and result interpretation.

# References


[1] J. Zhou et al., "Instruction-Following Evaluation for Large Language Models," *arXiv*:2311.07911, 2023.

[2] Y. Jiang et al., "FollowBench: A Multi-level Fine-grained Constraints Following Benchmark for Large Language Models," *Proc. ACL 2024*, pp. 4667–4688.

[3] Z. Guo et al., "RECAST: Expanding the Boundaries of LLMs' Complex Instruction Following with Multi-Constraint Data," *arXiv*:2505.19030, 2025.

[4] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *ICLR 2022*. *arXiv*:2106.09685.

[5] R. Shuttleworth et al., "LoRA vs Full Fine-tuning: An Illusion of Equivalence," *arXiv*:2410.21228, 2024.

[6] H. Ivison et al., "Camels in a Changing Climate: Enhancing LM Adaptation with Tulu 2," *arXiv*:2311.10702, 2023.

[7] J. Hoffmann et al., "Training Compute-Optimal Large Language Models," *NeurIPS 2022*.


# Appendix: AI Assistance Disclosure

Per course policy, this report was written with the assistance of Claude Sonnet 4.6 (Anthropic, 2026) for LaTeX formatting, prose editing, and structural organization. Prompts instructed the model to reformat the existing content into a two-column conference paper style (NeurIPS/ICML template), condense verbose descriptions while preserving all experimental results, and add team contributions. All experimental results, numbers, figures, and scientific conclusions were produced by the team and not generated by AI.